AGILE METHODOLOGY

Applied to Intrusion Detection Systems Using Machine Learning

Project report based on the uploaded IDS report using the UNSW-NB15 dataset.

Project domain	Network security and machine-learning-based intrusion detection
Dataset	UNSW-NB15
Methods in source report	Logistic Regression, Random Forest, Naive Bayes, Decision Tree, Bagging, XGBoost, Stacking, and Voting
Purpose of this document	Proposed Agile sprint plan, testing strategy, deliverables, and README documentation

1. Introduction

Agile is an iterative development approach in which a project is delivered in small increments, reviewed frequently, and improved through feedback. For an intrusion detection system (IDS), Agile supports experimentation with preprocessing methods, machine-learning models, and evaluation metrics while keeping the work transparent and testable.

The uploaded report evaluates machine-learning methods on the UNSW-NB15 dataset. It describes data preparation, feature encoding and scaling, handling class imbalance with SMOTE, and comparison of individual and ensemble classifiers. The Agile plan in this document is a recommended way to organize that work; it does not imply that the original project already followed formal sprints.

2. Project objectives

• Prepare the UNSW-NB15 data for modelling.
• Classify network traffic as attack or normal.
• Compare individual classifiers and ensemble methods.
• Evaluate accuracy, precision, recall, and F1-score.
• Improve detection of minority attack classes while monitoring false alarms.
• Produce a reproducible notebook and clear technical documentation.

3. Agile sprint plan

Sprint	Work items	Deliverable / review
1 — Requirements and data audit	Agree on binary target, success metrics, data permissions, and review criteria. Inspect files, columns, labels, class distribution, and data quality.	Requirements backlog and data-audit summary
2 — Data preparation	Implement duplicate checks, missing-value handling, categorical encoding, numeric conversion, feature selection, and scaling in a repeatable pipeline.	Reusable preprocessing pipeline and validation checks
3 — Baseline models	Train baseline Decision Tree and Logistic Regression models. Produce confusion matrices and class-level precision, recall, and F1-score.	Baseline comparison for review
4 — Ensemble models	Compare Random Forest, Bagging, XGBoost, Stacking, and Voting using consistent splits and metrics.	Model comparison table and updated backlog
5 — Class imbalance	Apply SMOTE only within the training workflow. Compare minority-class recall, precision, F1-score, and false-positive behaviour.	Imbalance-handling experiment results
6 — Validation and handover	Evaluate the selected model on the untouched test set, reproduce results, document limitations, and prepare the final notebook and instructions.	Validated report, reproducibility notes, and handover

4. Roles and collaboration

• Product owner/project coordinator: prioritizes requirements, maintains the backlog, and accepts sprint deliverables.
• Data analyst: audits the data and documents its structure and limitations.
• Machine-learning developer: builds preprocessing pipelines and trains models.
• Evaluation lead: defines metrics, checks test separation, and verifies reproducibility.
• Security/domain reviewer: assesses operational usefulness, false-alert consequences, and data-handling requirements.

5. Testing and quality assurance

• Validate columns, types, labels, missing values, and duplicate records.
• Fit encoders, scalers, and resampling steps using training data only.
• Apply SMOTE only after splitting, and only to the training portion, to avoid test leakage.
• Use consistent train/test partitions when comparing models.
• Report per-class precision, recall, F1-score, and confusion matrix, not accuracy alone.
• Pay special attention to rare classes such as Worms and Shellcode.
• Record random seeds, parameters, library versions, and dataset version so results can be reproduced.

6. Results referenced in the source report

The uploaded report states that Random Forest achieved accuracy of 0.9168 and F1-score of 0.9273 in its binary-detection comparison. It also describes Decision Tree, XGBoost, Stacking, Voting, and Bagging as strong approaches. These are source-reported results and have not been independently rerun while preparing this Agile document.

7. Definition of done

A sprint increment is complete when the code runs, tests pass, evaluation outputs are recorded, changes are reviewed, and the documentation is updated. The overall project is ready for handover when preprocessing is reproducible, test data remains untouched during model selection, metrics are reported clearly, and limitations are documented.

8. Risks and mitigations

• **Data leakage:** use a clean pipeline and keep SMOTE within training data.
• **Class imbalance:** evaluate rare classes and do not rely on accuracy alone.
• **Changing requirements:** keep a prioritized backlog and review each increment.
• **Unreproducible results:** record versions, random seeds, parameters, and data splits.
• **Deployment gap:** benchmark performance on representative live traffic and plan drift monitoring before operational use.

9. Conclusion

Agile can organize the IDS project into short, reviewable increments: audit data, prepare features, establish baselines, compare ensembles, address class imbalance, and validate the final model. Frequent review helps the team improve detection performance while keeping model evaluation, reproducibility, and operational risks visible.

Source note: Project-specific dataset, methods, and results are based on the uploaded report “Intrusion Detection Systems Using Machine Learning.” The sprint plan and Agile roles are proposed recommendations.